Guia Prático para Criação de Diagramas UML (Sistema Nov11 Sports)

Este guia detalha como criar os principais diagramas UML relevantes para o seu relatório, com foco no sistema Nov11 Sports. Serão abordados o Diagrama de Casos de Uso, o Diagrama de Classes e o Diagrama de Componentes. Além disso, serão sugeridas ferramentas e dicas para um design eficaz.

1. Ferramentas Recomendadas para Criação de Diagramas UML

Existem diversas ferramentas, gratuitas e pagas, que facilitam a criação de diagramas UML. Algumas das mais populares e acessíveis incluem:

- **Lucidchart:** Ferramenta online colaborativa com uma vasta biblioteca de formas UML. Oferece uma versão gratuita com funcionalidades limitadas. Lucidchart
- **draw.io (agora diagrams.net):** Ferramenta online gratuita e de código aberto, com integração com Google Drive, OneDrive, Dropbox, etc. É uma excelente opção para diagramas UML. diagrams.net
- **Visual Paradigm Community Edition:** Uma versão gratuita de uma ferramenta profissional, ideal para quem busca funcionalidades mais avançadas. Visual Paradigm
- **PlantUML:** Uma ferramenta baseada em texto que gera diagramas a partir de descrições simples. Ótima para versionamento e automação. Pode ser integrada em editores de texto e IDEs. PlantUML

Para este guia, as explicações serão genéricas o suficiente para serem aplicadas em qualquer uma destas ferramentas.

2. Como Criar o Diagrama de Casos de Uso

O Diagrama de Casos de Uso descreve as funcionalidades do sistema do ponto de vista do utilizador. Ele mostra os **atores** (quem interage com o sistema) e os **casos de uso** (o que o sistema faz).

Conceitos Chave:

- **Ator:** Representa um papel desempenhado por um utilizador ou outro sistema que interage com o sistema em estudo. No Nov11 Sports, temos: **Visitante**, **Utilizador Autenticado** e **Administrador**.

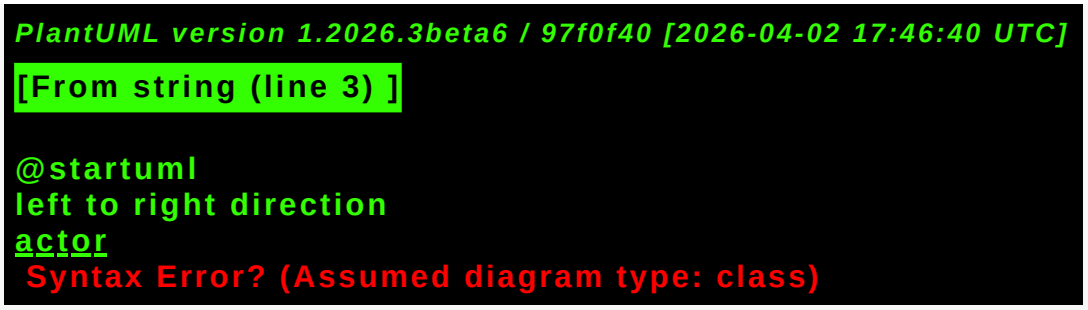
- **Caso de Uso:** Uma sequência de ações que o sistema executa para produzir um resultado observável de valor para um ator específico. Exemplos: `Comprar Bilhete`, `Agendar Estádio`, `Criar Evento`.
- **Relacionamentos:**
  - **Associação:** Conecta um ator a um caso de uso.
  - **Include:** Um caso de uso inclui a funcionalidade de outro caso de uso (obrigatório). Ex: `Comprar Bilhete` `<<include>>` `Fazer Login`.
  - **Extend:** Um caso de uso estende o comportamento de outro caso de uso (opcional). Ex: `Gerar Relatório` `<<extend>>` `Exportar PDF`.

Passo a Passo para o Sistema Nov11 Sports:

1. **Identificar Atores:** Comece por listar os atores principais: Visitante, Utilizador Autenticado, Administrador.
2. **Identificar Casos de Uso:** Para cada ator, liste as funcionalidades que ele pode realizar no sistema. Use os Requisitos Funcionais (RF) identificados anteriormente como base.
   - **Visitante:** `Visualizar Eventos`, `Pesquisar Eventos`, `Ver Detalhes do Evento`, `Ver Galeria`.
   - **Utilizador Autenticado:** `Fazer Login`, `Registrar`, `Comprar Bilhete`, `Gerir Perfil`, `Exportar Bilhete PDF`, `Agendar Estádio`, `Ver Status Agendamento`, `Contactar Suporte`.
   - **Administrador:** `Fazer Login (Admin)`, `Aceder Dashboard`, `Criar Evento`, `Gerir Agendamentos (Aceitar/Recusar)`, `Gerar Relatórios`, `Exportar Relatório PDF`, `Responder Suporte`, `Monitorizar Logs`.
3. **Desenhar o Diagrama:**
   - Desenhe um retângulo grande para representar os limites do sistema.
   - Coloque os atores fora do retângulo, representados por bonecos.
   - Coloque os casos de uso dentro do retângulo, representados por elipses.
   - Desenhe linhas de associação entre os atores e os casos de uso com os quais interagem.
   - Adicione relacionamentos `<<include>>` ou `<<extend>>` quando apropriado. Por exemplo, `Comprar Bilhete` `<<include>>` `Fazer Login`.

Exemplo de Estrutura (PlantUML para ilustração):

```plantuml
```



3. Como Criar o Diagrama de Classes

O Diagrama de Classes é a espinha dorsal da modelagem orientada a objetos, mostrando a estrutura estática do sistema. Ele representa as classes, os seus atributos, métodos e os relacionamentos entre elas.

Conceitos Chave:

- **Classe:** Uma representação de um conjunto de objetos com características (atributos) e comportamentos (métodos) semelhantes. Ex: `Utilizador`, `Evento`, `Agendamento`.
- **Atributo:** Uma propriedade de uma classe que descreve uma característica dos objetos dessa classe. Ex: `nome` (da classe `Utilizador`), `data` (da classe `Evento`).
- **Método (Operação):** Uma função ou procedimento que uma classe pode executar. Ex: `fazerLogin()` (da classe `Utilizador`), `criarEvento()` (da classe `Evento`).
- **Visibilidade:** Indica o nível de acesso a atributos e métodos:
  - `+` (Público): Acessível de qualquer lugar.
  - `-` (Privado): Acessível apenas dentro da própria classe.
  - `#` (Protegido): Acessível dentro da classe e suas subclasses.
- **Relacionamentos:**
  - **Associação:** Uma ligação estrutural entre objetos de diferentes classes. Pode ter cardinalidade (1 para 1, 1 para muitos, muitos para muitos).
  - **Agregação:** Um tipo de associação

    onde uma classe é parte de outra, mas pode existir independentemente (relação "tem um").
  - **Composição:** Um tipo forte de agregação onde a parte não pode existir sem o todo. Ex: Um `Bilhete` não existe sem um `Evento`.
  - **Generalização/Especialização (Herança):** Uma classe (subclasse) herda características de outra (superclasse). Ex: `Administrador` pode ser uma especialização de `Utilizador`.

Passo a Passo para o Sistema Nov11 Sports:

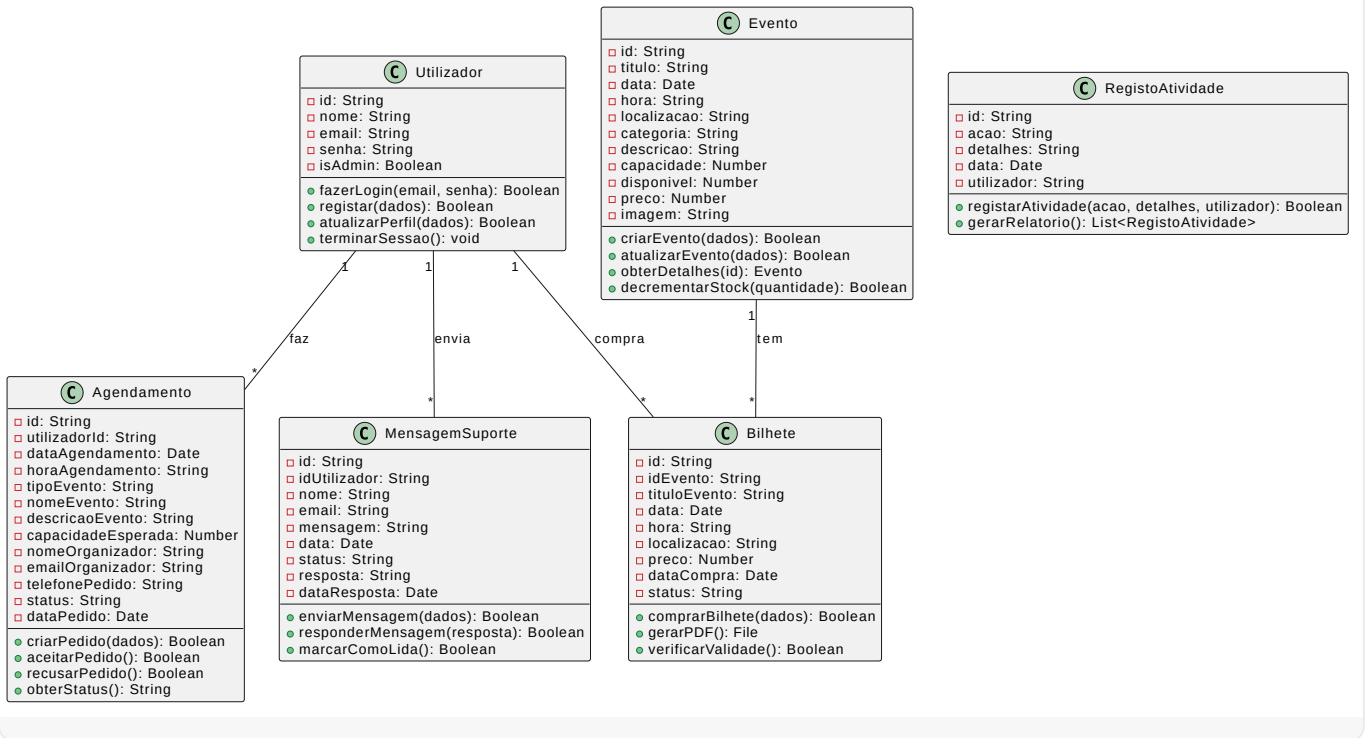
1. **Identificar Classes:** Use a tabela de classes que já definimos como ponto de partida: `Utilizador`, `Evento`, `Bilhete`, `Agendamento`, `MensagemSuporte`, `RegistoAtividade`.
2. **Definir Atributos e Métodos:** Para cada classe, liste os atributos (com tipo e visibilidade) e os métodos (com visibilidade e parâmetros, se houver). Por exemplo:
   - **Utilizador:**
     - `- id: String`
     - `- nome: String`
     - `- email: String`
     - `- senha: String`
     - `- isAdmin: Boolean`
     - `+ fazerLogin(email, senha): Boolean`
     - `+ registar(dados): Boolean`
     - `+ atualizarPerfil(dados): Boolean`
   - **Evento:**
     - `- id: String`
     - `- titulo: String`
     - `- data: Date`
     - `- hora: String`
     - `- localizacao: String`
     - `- categoria: String`
     - `- descricao: String`
     - `- capacidade: Number`
     - `- disponivel: Number`
     - `- preco: Number`
     - `- imagem: String`
     - `+ criarEvento(dados): Boolean`
     - `+ obterDetalhes(id): Evento`
     - `+ decrementarStock(quantidade): Boolean`
3. **Identificar Relacionamentos:** Analise como as classes interagem entre si. Por exemplo:
   - Um `Utilizador` pode ter muitos `Bilhetes` (1 para muitos).

- Um `Evento` pode ter muitos `Bilhetes` (1 para muitos).
- Um `Utilizador` pode fazer muitos `Agendamentos` (1 para muitos).
- Um `Agendamento` é feito por um `Utilizador` (muitos para 1).
- Um `RegistoAtividade` pode estar associado a um `Utilizador` (muitos para 0 ou 1).

4. **Desenhar o Diagrama:**
   - Represente cada classe como um retângulo dividido em três secções: nome da classe, atributos e métodos.
   - Conecte as classes com as linhas de relacionamento apropriadas, indicando a cardinalidade em cada extremidade.
   - Use setas para indicar a direção da navegação ou herança.

Exemplo de Estrutura (PlantUML para ilustração):

plantuml



4. Como Criar o Diagrama de Componentes

O Diagrama de Componentes mostra a estrutura de alto nível do sistema, focando nos componentes de software e nas suas interfaces e dependências. É ideal para visualizar a arquitetura do sistema.

Conceitos Chave:

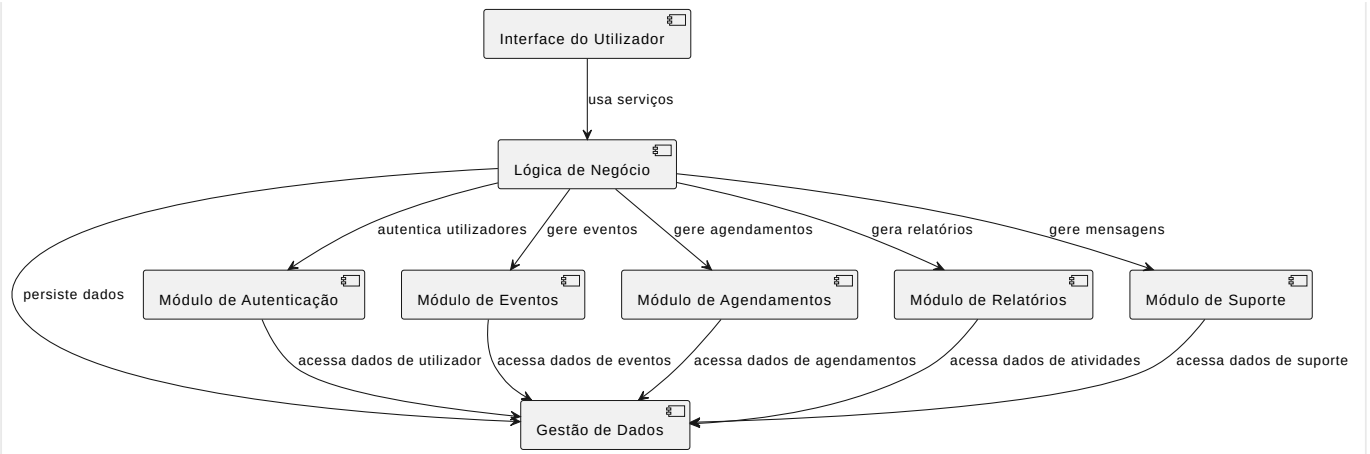
- **Componente:** Uma parte modular e substituível de um sistema que encapsula o seu conteúdo e expõe interfaces. No nosso caso, podemos pensar em módulos lógicos como "Interface do Utilizador", "Lógica de Negócio", "Gestão de Dados".
- **Interface:** Um conjunto de operações que um componente oferece aos outros (interface provida) ou que espera de outros (interface requerida).
- **Dependência:** Indica que uma mudança num componente pode afetar outro componente.

Passo a Passo para o Sistema Nov11 Sports:

1. **Identificar Componentes Principais:** Com base na arquitetura de três camadas mencionada no relatório (Apresentação, Lógica de Aplicação, Dados) e nas funcionalidades, podemos definir:
   - `Interface do Utilizador (Frontend)`
   - `Lógica de Negócio (Backend Lógico)`
   - `Gestão de Dados (Persistência)`
   - `Módulo de Autenticação`
   - `Módulo de Eventos`
   - `Módulo de Agendamentos`
   - `Módulo de Relatórios`
   - `Módulo de Suporte`
2. **Definir Interfaces:** Pense nas funcionalidades que cada componente oferece ou precisa. Por exemplo, a `Interface do Utilizador` provê a interação e requer serviços da `Lógica de Negócio`.
3. **Identificar Dependências:** Como os componentes interagem? A `Lógica de Negócio` depende da `Gestão de Dados` para armazenar e recuperar informações.
4. **Desenhar o Diagrama:**
   - Represente cada componente como um retângulo com um pequeno retângulo na parte superior direita ou com o ícone de componente.
   - Use linhas com setas para indicar as dependências e interfaces.

Exemplo de Estrutura (PlantUML para ilustração):

```plantuml
```



5. Dicas de Design e Boas Práticas

- **Simplicidade:** Mantenha os diagramas o mais simples e claros possível. Evite sobrecarregar com muitos detalhes.
- **Consistência:** Use uma notação consistente para todos os elementos e relacionamentos.
- **Legibilidade:** Garanta que o texto seja legível e que as linhas não se cruzem desnecessariamente.
- **Foco:** Cada diagrama deve ter um propósito claro e focar-se num aspeto específico do sistema.
- **Revisão:** Peça a outras pessoas para reverem os seus diagramas para garantir que são compreensíveis e precisos.
- **Contexto:** Sempre forneça uma breve introdução e explicação para cada diagrama no seu relatório.

Espero que este guia detalhado o ajude a criar os seus diagramas UML com confiança e precisão para o seu relatório!